
Lessons from Sim-to-Real on a 15-DOF Bipedal Robot: Hardware Realities, Reward Brittleness, and the Limits of BC Diffusion for Locomotion

Zhenghao Xu

Elmore Family School of Electrical and Computer Engineering
Purdue University
West Lafayette, IN 47906
xu1783@purdue.edu

Abstract

We present a reflective account of working with the Open Duck Mini V2, a low-cost 15-DOF bipedal robot. The work spanned hardware assembly, PPO policy training in MuJoCo MJX, sim-to-real transfer attempts, and an application of Diffusion Policy via behavior cloning from PPO rollouts. The central outcomes are negative: a PPO policy that dominates in simulation (beating the author-provided baseline on 11/14 metrics) fails to transfer to hardware, and a BC diffusion policy fails catastrophically even in simulation (98% fall rate). Rather than presenting these as successes, we analyze where each stage broke down. We identify three root causes: (1) hardware assembly errors that created unmodeled dynamics, (2) reward overfitting in simulation without real-world validation, and (3) the inadequacy of offline BC-from-distillation for unstable bipedal locomotion without corrective feedback. We conclude with concrete recommendations—system identification, real-world data collection, and DPPO—that would address each failure mode.

1 Introduction

Sim-to-real transfer remains one of the central challenges in robot learning. The promise is compelling: train policies endlessly in simulation at zero marginal cost, then deploy on hardware. In practice, the gap between simulated and real dynamics—the *sim-to-real gap*—makes this transfer fragile, especially for inherently unstable systems like bipedal robots.

This paper recounts a hands-on project with the Open Duck Mini V2, a small, low-cost (~\$200) bipedal robot with 15 degrees of freedom. The work progressed through four stages:

1. **Hardware assembly** of the physical robot from a kit
2. **PPO policy training** in MuJoCo MJX, iterated over 28 experiments
3. **Sim-to-real transfer** attempts with the trained policies
4. **Diffusion Policy** via behavior cloning from PPO rollouts

Each stage produced a negative result or an instructive failure. The trained PPO policy beat the original author’s baseline on 11 of 14 simulation metrics but performed poorly on hardware. The BC diffusion policy achieved a 98% fall rate in simulation. Rather than presenting selective successes, we document these failures honestly and analyze their root causes. We believe this kind of failure analysis is underrepresented in the literature and is often more instructive than success stories.

Our contributions are:

- A detailed analysis of how hardware assembly errors (missing Loctite, mis-initialized joints) create sim-to-real gaps that domain randomization cannot close
- A case study in reward overfitting: a policy that dominates in simulation but fails on hardware

- An empirical demonstration that BC-from-distillation fails catastrophically on bipedal locomotion, with analysis of why—and how this differs from the manipulation setting where Diffusion Policy succeeds
- Concrete recommendations for future work: system identification, real2sim2real loops, and DPPO

2 Related Work

Sim-to-real transfer. Domain randomization [?] is the standard approach for bridging the sim-to-real gap, treating modeling errors as noise to be averaged over. System identification [?] takes the complementary approach of directly measuring and matching real-world dynamics. Our experience suggests that for bipedal robots with significant hardware variability, SysID is essential and randomization alone is insufficient.

Diffusion Policy. Diffusion Policy [?] applies denoising diffusion models to visuomotor policy learning, demonstrating strong performance on manipulation tasks. DPPO [?] extends this by using a diffusion policy as the policy class within a PPO objective, combining diffusion’s expressive action representation with RL’s corrective feedback loop.

Behavior cloning limitations. The compounding error problem in BC was formalized by [?]. We discuss this in detail in Section 6.3 in the context of our empirical results.

3 The Platform

The Open Duck Mini V2 is a 15-DOF bipedal robot: 10 leg joints (hip yaw/roll/pitch, knee, ankle per leg) and 5 head/neck joints. It runs on a Raspberry Pi 5 with Feetech STS3215 servos and an onboard IMU. The upstream repository provides a complete PPO training pipeline using Brax in MuJoCo MJX, with automatic ONNX export and a runtime for the Pi.

The robot’s appeal is its low cost and open-source toolchain. The challenge is that it is a biped—inherently less stable than a quadruped, and far more sensitive to sim-to-real gaps in joint dynamics, mass distribution, and sensor calibration.

The observation space is 101-dimensional: gyroscope (3), accelerometer (3), velocity command (7), joint angles minus defaults (14), joint velocities (14), three steps of action history (3×14), motor targets (14), foot contacts (2), and gait phase (2). The action space is 14-dimensional: position offsets applied as $\text{motor_targets} = \text{default} + \text{action} \times 0.25$. The policy runs at 50 Hz (20 ms control period) with physics at 500 Hz.

4 Hardware Lessons

4.1 Missing Loctite on Actuator Screws

The assembly guide did not emphasize that threadlocker (Loctite) is essential for all screws on the actuator joints. After several sessions of testing, screws on multiple actuators loosened progressively, introducing mechanical play—backlash and friction that the simulated model did not account for.

The Feetech servos produce enough torque to vibrate screws loose over hundreds of gait cycles. The resulting backlash is asymmetric, velocity-dependent, and differs from joint to joint. The backlash simulation in MuJoCo (`scene_flat_terrain_backlash.xml`) models generic gear play but cannot capture this specific pattern of screw-loosening-induced play.

Lesson. Always use threadlocker on any joint that transmits force. This is a hardware problem that demands a hardware fix, not a software one.

Table 1: PPO policy comparison: custom policy (28 experiments of reward tuning) vs. the original author’s best policy. The custom policy beats the baseline on 11/14 metrics in simulation.

Metric	Custom Policy	Author Baseline	Delta
Stand tracking (angular)	0.907	0.883	+2.7%
Forward tracking (angular)	0.655	0.520	+26%
Backward tracking (linear)	0.229	0.073	+214%
Turn right (angular)	0.526	0.286	+84%
Turn left (angular)	0.547	0.322	+70%
Metrics improved	11 / 14		

4.2 Incorrect Neck Joint Initialization

The assembly guide did not provide clear instructions for initializing the neck joints (`neck_pitch`, `head_pitch`, `head_yaw`, `head_roll`) to their zero positions. We suspect the joints were initialized at an offset from where the simulated model expects them.

For a biped, this matters more than it might seem. The head assembly has non-trivial mass, and its position affects the center of mass of the entire robot. On a quadruped, a small COM offset is a nuisance. On a biped, it can be the difference between balancing and falling.

Lesson. Before any sim-to-real deployment, verify every joint’s zero position against the simulated model’s home keyframe. The runtime’s `init_pos` array must exactly match `keyframe("home").ctrl` from the MJCF model.

5 PPO Policy Training

5.1 Training Pipeline

We trained PPO policies using the MuJoCo MJX pipeline on rented cloud GPUs (Vast.ai, RTX 4060 Ti). Over 28 experiments, we iterated on reward weights through stacked fine-tuning: starting from a baseline policy, progressively adjusting reward scales and continuing training from the best checkpoint.

The key parameter discovery was that `tracking_ang_vel` dominates forward walking quality. Increasing it from the default 6.0 to 14.0 (through a chain of $6 \rightarrow 8 \rightarrow 10 \rightarrow 12 \rightarrow 14$) steadily improved angular velocity tracking and forward gait, at the cost of lateral tracking. The best policy (experiment 28, 159M steps) achieved the results shown in Table 1.

5.2 The Sim-to-Real Gap

Despite dominating in simulation, the custom policy performed poorly on the real robot. The robot could stand and hold position, but walking was unstable—it would fall after a few steps, especially during turns and lateral movement.

In retrospect, this is a classic case of *reward overfitting in simulation*. By aggressively optimizing `tracking_ang_vel` through 28 experiments, we produced a policy that exploits the simulated dynamics to achieve high reward scores. The policy learned behaviors that are brittle: they work in the narrow band of dynamics the simulator provides but fail when the real dynamics diverge.

The original author’s policy, while scoring lower in simulation, was validated on real hardware during development. It learned a more conservative gait that is robust to the sim-to-real gap. Our policy learned a more aggressive gait that scores higher in sim but lacks this robustness.

Lesson. Beating an author-provided baseline in simulation does not mean you have a better policy for the real robot. If your goal is sim-to-real transfer, evaluate on hardware early and often. Simulation metrics are necessary but not sufficient.

Table 2: Sim evaluation: PPO baseline vs. BC diffusion policy. The diffusion policy achieves 98% fall rate and survives only 9 steps on average.

Metric	PPO Baseline	Diffusion BC	Delta
Avg survival steps	100 / 1000	9 / 1000	−91%
Fall rate	0%	98%	+98 pp
Stand tracking (linear)	0.809	0.125	−85%
Action smoothness (rate)	0.053	0.357	6.7× noisier

5.3 Domain Randomization Was Not Enough

Our primary strategy for sim-to-real transfer was domain randomization during training: varying floor friction, joint PID gains, mass distribution, initial poses, and applying random pushes.

The problem is that domain randomization treats the sim-to-real gap as *zero-mean noise* around the simulated parameters. It assumes the real robot’s dynamics are somewhere within the randomization envelope. But when the real robot has systematic biases—loosened screws creating asymmetric backlash, a shifted center of mass, servo responses that differ from the identified model—no amount of randomization centered on the wrong parameters will help. You cannot randomize your way out of a systematic modeling error.

6 Diffusion Policy via Behavior Cloning

6.1 Motivation and Setup

We applied Diffusion Policy [?] to this platform via behavior cloning from PPO rollouts: generate (o_t, a_t) pairs by rolling out the PPO policy in simulation, then train a Conditional U-Net to predict action chunks via denoising.

- **Dataset:** 4,755 trajectories from the original author’s PPO policy, collected at 50 Hz with randomized velocity commands, friction, initial poses, and impulse perturbations. Total: 865,780 steps.
- **Model:** ConditionalUnet1D (16.4M parameters) with DDIM sampling (16 denoising steps). Action chunk size 16, observation horizon 2.
- **Training:** 100 epochs, batch size 256, AdamW with cosine LR. Final validation loss: 0.0221.

6.2 Results

The diffusion policy failed catastrophically. Table 2 shows the head-to-head comparison.

Only 20% of episodes survived even the `stand` command. All motion commands caused immediate falls.

6.3 Analysis: Why BC Diffusion Fails on Bipedal Locomotion

Compounding error (distribution shift). BC trains a mapping from observations to actions using expert data. At test time, any small prediction error causes the state to leave the training distribution. The next observation is now out-of-distribution, producing a worse action, compounding until the robot falls. This is the DAGger problem [?]: BC policies are correct only on states visited by the expert, but must act on states produced by their own imperfect predictions.

Action smoothness. The diffusion policy’s action rate was 5–50× higher than PPO’s. Even with DDIM’s deterministic sampling, the denoised action chunks had high inter-step variance. For a bipedal robot that requires smooth, coordinated joint trajectories, jerky actions are immediately destabilizing.

Unimodal teacher. Distilling from a single PPO policy produces a unimodal action distribution. The diffusion model only sees one way to respond to each situation and cannot learn the diverse

recovery behaviors that PPO implicitly uses to stay upright. PPO developed these through the RL survival reward; the diffusion policy never experienced this pressure.

Locomotion vs. manipulation. This failure highlights a fundamental distinction between locomotion and manipulation. Diffusion Policy was demonstrated on manipulation tasks where small action errors are recoverable—a slightly misaligned gripper can be corrected on the next step. In bipedal locomotion, the state space is far less forgiving: a small error in foot placement can cause an unrecoverable fall within one or two steps. This is precisely why DPPO [?], which combines diffusion’s expressive action representation with RL’s corrective feedback, is the natural next step for locomotion.

7 What We Would Do Differently

7.1 System Identification Before Training

The single most impactful change would be system identification (SysID) on the real robot before training. Instead of broad domain randomization centered on generic parameters, we would:

1. Measure each joint’s actual response to commanded positions at various speeds and loads
2. Identify the actual backlash profile per joint (likely asymmetric and joint-specific after screw loosening)
3. Measure the real robot’s mass and COM by physical weighing
4. Update the MuJoCo model’s joint parameters, friction coefficients, and body masses to match

Domain randomization is a workaround for an inaccurate model; SysID fixes the model.

7.2 Real-World Data Collection and Fine-Tuning

With more time, we would set up a data collection pipeline on the real robot:

1. Run the author’s best policy on the real robot while recording observations and actions at 50 Hz
2. Use this real-world data to fine-tune via offline RL or to identify systematic observation–action mismatches
3. Iterate: collect data → update model → retrain → evaluate → repeat

This is the “real2sim2real” loop that has become standard in modern robot learning. Even a small amount of real-world data is more valuable for closing the sim-to-real gap than any amount of simulated data.

7.3 DPPO Instead of BC Diffusion

For the diffusion policy work, the correct approach would be DPPO [?], which uses a diffusion model within an on-policy RL objective. This addresses both the compounding error problem (RL provides corrective feedback) and the unimodal teacher limitation (the policy explores diverse recovery behaviors). BC-from-distillation is a reasonable starting point for manipulation where errors are non-catastrophic, but it is the wrong tool for bipedal locomotion.

8 Conclusion

This project produced two negative results: a PPO policy that excels in simulation but fails on hardware, and a BC diffusion policy that fails even in simulation. Both failures are informative.

The sim-to-real gap was dominated by hardware issues (loose screws, mis-initialized joints) that domain randomization cannot fix. The correct response to a systematic sim-to-real gap is not more randomization but better modeling—system identification to bring the simulated dynamics closer to reality.

The diffusion policy failure is a textbook illustration of behavior cloning’s limitations on unstable systems. It reinforces the principle that locomotion requires closed-loop corrective feedback, whether

from RL (PPO, DPPO) or from real-world data. BC is well-suited for tasks where small errors are recoverable; bipedal walking is not such a task.

The most valuable outcome of this project is not any single result but the understanding of *where* each method breaks down and *why*. For future work, the path forward is clear: SysID to close the sim-to-real gap, real2sim2real loops for iterative refinement, and DPPO to combine diffusion’s expressive action representation with RL’s corrective feedback for bipedal locomotion.

9 Outlook

This project concluded earlier than intended due to time constraints. However, the lessons learned here directly inform ongoing and planned work. The immediate next step is a collaborative project on hierarchical multi-agent policies for a team of four Open Duck robots, where the sim-to-real insights and policy-class comparisons from this warm-up project will be applied in a multi-agent setting. Beyond this, the core takeaways—that system identification matters more than domain randomization, that closed-loop feedback is essential for unstable systems, and that honest failure analysis accelerates learning—transfer across embodiments to upcoming work on quadrupedal locomotion and bimanual manipulation.

Acknowledgments and Disclosure of Funding

We thank the Open Duck Mini community and the original author (Antoine Pirrone) for the open-source hardware design, training pipeline, and reference policies.